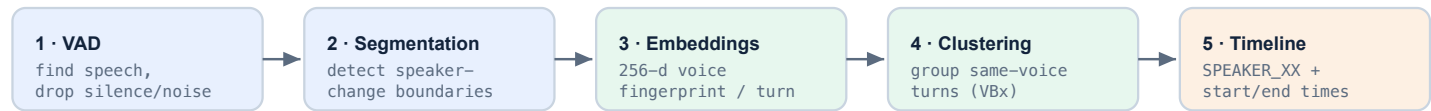


Offline Speaker Diarization — Pipeline Explainer

Answer “who spoke when” for any recorded audio/video · open-source · GPU or CPU · Windows / Linux / macOS

What it does: take a whole recording and output a **speaker-labeled timeline** — each moment tagged `SPEAKER_00`, `SPEAKER_01`, ... — with the same voice always getting the same label, and **no fragmentation** (one person is never split into phantom speakers).

THE PIPELINE (5 STEPS)



A neural model does steps 1–3; clustering (4) groups the voice fingerprints with a *global view of the whole file*.

KEY CONCEPTS

Voice fingerprints separate cleanly

- Each speech turn → a 256-dim **embedding**.
- Same speaker ≈ **0.83** cosine similarity; different speakers ≈ **0.10** — a wide, reliable gap.
- Clustering exploits that gap to assign consistent labels.

Why it doesn’t fragment

- Global clustering sees the **whole file**, not tiny chunks.
- An expressive speaker’s varied turns still cluster together.
- Two safeguards clean the edges (below).

THE TWO ACCURACY SAFEGUARDS

Safeguard	Problem it fixes	Knob
Clustering threshold	one expressive speaker over-split into extra IDs (raise) / two similar voices merged (lower)	<code>--threshold 0.85</code>
Small-cluster consolidation	phantom speakers formed from noisy 1–2 s interjections → merged into nearest real voice	<code>--min-speaker-dur 20</code>
Known count (best)	removes all guesswork when the format is fixed (e.g. 2-host podcast)	<code>--speakers 2</code>

RUN IT

`python verify.py interview.mp3 --speakers 2` → prints the speaker/change summary, writes `verify_result.json`, and opens a colored `timeline.html`.

Library: `from diarizer import diarize_file; timeline, n = diarize_file("audio.wav", num_speakers=2)`

NOTES FOR REPRODUCTION

- GPU:** NVIDIA CUDA (or Apple MPS) is ~10–30× faster than CPU; install the matching PyTorch build.
- Model access:** free Hugging Face token + accept the `pyannote/speaker-diarization-community-1` terms once; weights (~1–2 GB) cache locally, then run offline.
- Engine:** `pyannote-audio 4 · community-1 · wespeaker embeddings · VBx clustering · PyTorch` — all open-source, no cloud, no paid APIs.
- Honest limitation:** for unknown speaker counts, no single threshold is perfect for every recording — prefer `--speakers N` when the count is known.